

Linux: системные интерфейсы и сбор телеметрии

Практический справочник для будущего Linux Agent в AI PC Guardian

/proc	/sys	syscalls	systemd
-------	------	----------	---------

Что даст этот справочник

После чтения ты поймешь, какие данные брать из `/proc` и `/sys`, где использовать `syscall/library API`, почему не надо парсить вывод `top`, и как встроить Linux Agent в общую модель проекта.

Платформа: Linux + systemd | Редакция: 01.09.2026 | Язык примеров: C++17

Материал привязан к архитектуре AI PC Guardian. Это учебная выжимка, а не замена официальной документации API.

Содержание

1. Что именно строит AI PC Guardian	3
Термины: переменные окружения и системные метрики	3
Контракт одного измерения	3
2. Как Linux отдает системные данные	4
3. File descriptor, errno и RAII	4
4. Переменные окружения и XDG-каталоги	5
5. CPU через /proc/stat	5
6. RAM через /proc/meminfo	6
7. Uptime и load average	6
8. Диски и файловые системы	6
9. Сеть	7
10. Процессы через /proc/PID	7
11. Температуры и hardware monitoring	8
12. Идентичность ОС и устройства	8
13. Unix domain socket вместо Named Pipe	8
14. Запуск как systemd service	8
15. Где находится Python и LLM	9
16. Рекомендуемая структура Linux-модуля	10
17. Тестирование Linux collector	10
18. Порядок реализации Linux-версии	10
19. Чек-лист code review	11
20. Официальные источники	11

1. Что именно строит AI PC Guardian

Клиент под Linux - это локальный агент. Он регулярно читает измеримые факты у операционной системы, приводит их к общей модели `guardian::common`, сохраняет последний снимок, сериализует сообщения через Protobuf и отправляет их локальному интерфейсу или серверу. Сетевой сервер уже решает, что хранить, агрегировать и передавать модели.



Короткий ответ на главный вопрос

На Linux не надо запускать `top`, `free`, `df` и разбирать их текст. Для MVP агент читает стабильные kernel-интерфейсы `/proc`, `/sys` и системные вызовы. Утилиты оставляем только для ручной диагностики. Python и LLM находятся на сервере; системный агент остается предсказуемым C++-процессом.

Термины: переменные окружения и системные метрики

Это две разные темы. Переменные окружения - строки конфигурации процесса: путь к каталогу данных, адрес сервера, уровень логирования. Системные метрики - CPU, RAM, диск, сеть, процессы и uptime, которые читаются через API ОС. В справочнике разобраны обе темы, но секреты и API-ключи не должны попадать в клиентскую телеметрию или логи.

Слой	Ответственность	Не должен делать
Agent / C++	Собирать факты, валидировать диапазоны, ставить timestamp, сериализовать	Интерпретировать диагноз с помощью LLM
Backend	Аутентификация, прием, хранение, агрегация, rate limit	Доверять любому полю клиента без проверки
AI service / Python	Редакция контекста, prompt, вызов модели, проверка ответа	Получать полный дамп системы без минимизации данных
GUI	Показывать состояние, ошибки и рекомендации	Блокировать поток сборщика

Контракт одного измерения

- Считывай счетчики дважды, если метрика является скоростью или процентом: CPU, bytes/sec, packets/sec.
- Храни сырые величины в целых числах и явно фиксируй единицы: bytes, milliseconds, percent.
- Отмечай источник и ошибку отдельным полем; 0 не должен означать одновременно реальный ноль и сбой.
- Используй monotonic clock для интервалов и system clock для timestamp события.
- Сборщик не бросает исключение наружу на обычном отказе доступа: он возвращает ошибку и продолжает собирать другие метрики.

2. Как Linux отдает системные данные

Linux не имеет одного монолитного аналога WinAPI. Метрики приходят из нескольких стабильных интерфейсов: виртуальная файловая система `/proc` экспортирует состояние ядра и процессов, `/sys` - устройства и их атрибуты, системные вызовы возвращают данные через `libc`, а `netlink` дает структурированные сообщения о сети.

Источник	Использовать для	Пример
<code>/proc</code>	CPU, RAM, uptime, процессы, counters	<code>/proc/stat</code> , <code>/proc/meminfo</code>
<code>/sys</code>	Устройства, sensors, topology	<code>/sys/class/hwmon</code>
<code>libc/syscall</code>	Файловая система, uname, интерфейсы	<code>statvfs</code> , <code>uname</code> , <code>getifaddrs</code>
<code>netlink</code>	Сеть и события ядра	<code>NETLINK_ROUTE</code>
D-Bus	Управление системными сервисами	systemd APIs, если понадобится

Почему не `top | grep`

CLI-вывод рассчитан на человека: локаль, формат, набор колонок и округление меняются. Запуск subprocess тяжелее, создает новые ошибки и усложняет безопасность. Для диагностики команды полезны, но collector должен читать первичный интерфейс.

3. File descriptor, errno и RAII

Linux syscall обычно возвращает `-1` при ошибке и записывает код в thread-local `errno`. Сохрани `errno` сразу. File descriptor - небольшое целое, принадлежащее процессу; его закрывают `close`. В современном C++ заверни fd в move-only RAII-класс.

```
#include <cerrno>
#include <system_error>
#include <utility>
#include <unistd.h>

class unique_fd {
public:
    explicit unique_fd(int fd = -1) noexcept : fd_(fd) {}
    ~unique_fd() { if (fd_ >= 0) ::close(fd_); }
    unique_fd(const unique_fd&) = delete;
    unique_fd& operator=(const unique_fd&) = delete;
    unique_fd(unique_fd&& other) noexcept
        : fd_(std::exchange(other.fd_, -1)) {}
    int get() const noexcept { return fd_; }
private:
    int fd_;
};
```

- Short read/write нормальны: повторяй операцию до нужного размера или terminal error.
- `EINTR` означает, что syscall прерван сигналом; обычно вызов повторяют.
- `EACCES/EPERM` при чтении чужого процесса - ожидаемый частичный отказ.
- Проверь overflow при переводе pages, KiB и ticks в bytes/milliseconds.

4. Переменные окружения и XDG-каталоги

`std::getenv` возвращает указатель на внутреннее значение окружения. Скопируй строку, если она нужна дольше, и не модифицируй буфер. Для каталогов пользователя следуй XDG Base Directory: config, data, state и cache хранятся раздельно. У `systemd service` окружение задается в `unit/config`, а не берется из интерактивного shell.

```
#include <cstdlib>
#include <optional>
#include <string>

std::optional<std::string> env(const char* name) {
    if (const char* value = std::getenv(name)) {
        return std::string(value); // копия
    }
    return std::nullopt;
}

// Примеры приложения:
// GUARDIAN_SERVER_ADDRESS=127.0.0.1:50051
// GUARDIAN_LOG_LEVEL=info
```

Назначение	Переменная	Fallback
Конфигурация	<code>XDG_CONFIG_HOME</code>	<code>\$HOME/.config</code>
Постоянные данные	<code>XDG_DATA_HOME</code>	<code>\$HOME/.local/share</code>
Состояние/логи	<code>XDG_STATE_HOME</code>	<code>\$HOME/.local/state</code>
Кэш	<code>XDG_CACHE_HOME</code>	<code>\$HOME/.cache</code>
Runtime socket	<code>XDG_RUNTIME_DIR</code>	Задается login manager; не выдумывать в <code>/tmp</code> без защиты

5. CPU через /proc/stat

Первая строка `/proc/stat` начинается с `cpu` и содержит накопленные jiffies: user, nice, system, idle, iowait, irq, softirq, steal и другие поля. Процент считается по разности двух снимков. Для aggregate ratio значение `_SC_CLK_TCK` не нужно - единицы сокращаются; оно нужно при переводе ticks во время.

```
struct CpuTimes {
    uint64_t user{}, nice{}, system{}, idle{}, iowait{};
    uint64_t irq{}, softirq{}, steal{};
};

double cpu_percent(const CpuTimes& a, const CpuTimes& b) {
    auto idle = [](const CpuTimes& x) { return x.idle + x.iowait; };
    auto total = [](const CpuTimes& x) {
        return idle(x) + x.user + x.nice + x.system +
            x.irq + x.softirq + x.steal;
    };
    if (idle(b) < idle(a) || total(b) < total(a)) return 0.0;
    const uint64_t idle_delta = idle(b) - idle(a);
    const uint64_t total_delta = total(b) - total(a);
    if (total_delta == 0 || idle_delta > total_delta) return 0.0;
    return 100.0 * static_cast<double>(total_delta - idle_delta)
        / total_delta;
}
```

Парсер должен принимать дополнительные поля в конце строки, проверять numeric conversion и не предполагать фиксированную длину файла. Для per-core используй строки `cpu0`, `cpu1` и т.д.; hotplug CPU означает, что набор строк может измениться.

6. RAM через /proc/meminfo

Для пользовательского ощущения доступной памяти главные поля - `MemTotal` и `MemAvailable`. Значения обычно указаны в KiB, поэтому умножай на 1024 с проверкой overflow. Не используй `MemFree` как available: page cache может быть освобожден.

```
std::optional<MemorySample> parse_meminfo(std::istream& input) {
    std::string line;
    uint64_t value = 0, total_kib = 0, available_kib = 0;
    while (std::getline(input, line)) {
        std::istringstream row(line);
        std::string key;
        if (!(row >> key >> value)) continue;
        if (key == "MemTotal:") total_kib = value;
        else if (key == "MemAvailable:") available_kib = value;
    }
    if (total_kib == 0 || available_kib > total_kib) return std::nullopt;
    constexpr uint64_t kib = 1024;
    return MemorySample{total_kib * kib, available_kib * kib};
}
```

Парсинг `/proc`

Считай файлы небольшими snapshot-интерфейсами: открыть, прочитать целиком или последовательно, закрыть. Не держи stream открытым между измерениями и не считай порядок полей контрактом.

7. Uptime и load average

Метрика	Источник	Замечание
Uptime	<code>/proc/uptime</code> или <code>sysinfo(2)</code>	Monotonic duration, не wall-clock timestamp
Load average	<code>/proc/loadavg</code> или <code>getloadavg</code>	Не является процентом CPU
Timestamp	<code>std::chrono::system_clock</code>	Unix milliseconds для protobuf
Интервал	<code>std::chrono::steady_clock</code>	Не скачет при коррекции системных часов

Load average показывает среднее число runnable и uninterruptible задач. Значение 4.0 на машине с 4 CPU имеет другой смысл, чем на машине с 32 CPU. Не выдавай его пользователю как 400% CPU.

8. Диски и файловые системы

Для емкости конкретной точки монтирования используй `statvfs`. Полезный объем для непривилегированного пользователя - `f_bavail * f_frsize`, полный - `f_blocks * f_frsize`. Для I/O counters существует `/proc/diskstats`; это накопленные счетчики, поэтому throughput и IOPS считаются по дельте.

```
#include <sys/statvfs.h>

std::optional<DiskSpace> read_disk_space(const char* path) {
    struct statvfs s {};
    if (::statvfs(path, &s) != 0) return std::nullopt;

    const uint64_t block = static_cast<uint64_t>(s.f_frsize);
    return DiskSpace{
        static_cast<uint64_t>(s.f_blocks) * block,
        static_cast<uint64_t>(s.f_bavail) * block};
}
```

- Не суммируй bind mounts и container overlay как независимые физические диски.

- Идентификатор `/dev/sda` не универсален: NVMe выглядит как `/dev/nvme0n1`, virtual disks иначе.
- Связи mountpoint с device через `/proc/self/mountinfo`; учитывай namespaces.
- Disk health/SMART - отдельная привилегированная подсистема, не MVP collector.

9. Сеть

Для MVP можно прочитать counters из `/proc/net/dev`, но для устойчивого сетевого inventory и событий лучше rtnetlink. `getifaddrs` удобно возвращает список адресов и интерфейсов, однако Linux-specific статистика в `ifa_data` требует проверки типов.

Задача	Интерфейс	Когда применять
RX/TX bytes MVP	<code>/proc/net/dev</code>	Простой polling
Адреса интерфейсов	<code>getifaddrs</code>	Snapshot адресов
Статистика/маршруты	rtnetlink	Production и события
DNS	systemd-resolved/D-Bus или config	Отдельная диагностическая функция

```
struct NetworkCounters { uint64_t rx_bytes; uint64_t tx_bytes; };

double bytes_per_second(uint64_t before, uint64_t after,
                        std::chrono::duration<double> elapsed) {
    if (after < before || elapsed.count() <= 0.0) return 0.0;
    return static_cast<double>(after - before) / elapsed.count();
}

// Ключ snapshot: ifindex, не отображаемое имя.
// Loopback учитывай отдельно, иначе GUI покажет внутренний трафик как сеть.
```

10. Процессы через /proc/PID

Каждый видимый PID имеет каталог `/proc/PID`. `status` удобен для именованных полей, `stat` компактен для CPU/start time, `cmdline` содержит NUL-разделенные аргументы, `exe` - symlink. Процесс может исчезнуть после получения списка; ошибка `ENOENT` означает race и должна пропускаться.

Файл	Что брать	Осторожно
<code>status</code>	Name, State, Uid, VmRSS	Набор полей развивается
<code>stat</code>	utime, stime, starttime	Имя в скобках может содержать пробелы/)
<code>cmdline</code>	Аргументы	Может содержать секреты; часто недоступен
<code>exe</code>	Путь executable	Symlink может быть (deleted)
<code>smaps_rollup</code>	Сводная memory map	Дороже и ограничено правами

PID переиспользуется

PID недостаточен как долговечный идентификатор процесса. Для snapshot key используй пару (`pid`, `starttime`) из `/proc/PID/stat`; иначе новый процесс может получить PID старого.

11. Температуры и hardware monitoring

Датчики экспортируются через hwmon в `/sys/class/hwmon/hwmon*`. Типичное `temp1_input` содержит целое число в тысячных градуса Цельсия. Но labels и scaling зависят от драйвера; kernel documentation рекомендует userspace library вроде libsensors для корректных вычислений и конфигурации.

```
std::optional<double> millidegree_to_celsius(std::string_view raw) {
    int64_t value = 0;
    const auto [end, ec] = std::from_chars(
        raw.data(), raw.data() + raw.size(), value);
    if (ec != std::errc{}) return std::nullopt;
    return static_cast<double>(value) / 1000.0;
}

// MVP: sensor optional. Production: libsensors + label + chip identity.
```

12. Идентичность ОС и устройства

- `uname(2)`: kernel name/release/machine; это ядро, а не название дистрибутива.
- `/etc/os-release`: ID, VERSION_ID, PRETTY_NAME; parser обязан понимать quoted values.
- `/etc/machine-id`: стабильный machine identifier, но его нельзя напрямую отправлять наружу; хэшируй с app-specific key или предпочти собственный UUID.
- App-generated `DeviceId` хранится в XDG state/data каталоге и имеет одинаковую семантику с Windows.
- В контейнере сведения могут описывать container namespace, а не физический host - явно фиксируй deployment mode.

13. Unix domain socket вместо Named Pipe

Локальный аналог IPC - Unix domain socket (`AF_UNIX`). Для Qt GUI удобно использовать `QLocalServer/QLocalSocket`, а core transport может работать через POSIX socket. Формат сообщений оставляем одинаковым: 4-byte length prefix + Protobuf payload.

- Размещай socket в защищенном `XDGRUNTIME_DIR`, права файла обычно `0600` или group policy.
- Проверяй peer credentials (`SO_PEERCRED`) до обработки команд.
- Удаляй stale socket file только после проверки, что живого listener нет.
- Short read/write и max frame проверяются так же, как в Windows pipe.
- Не слушай TCP localhost для IPC без необходимости: Unix socket проще ограничить правами файловой системы.

14. Запуск как systemd service

На desktop MVP агент можно запускать вручную, затем через user service. Для системного daemon используется unit. `SIGTERM` означает корректную остановку; обработчик сигнала не должен вызывать сложный C++ код. Практичный вариант - блокировать сигналы и принимать их через `signalfd/event loop` либо выставить атомарный флаг и разбудить loop.

Черновой systemd unit. Реальные sandbox options проверяются интеграционным тестом, потому что collector должен читать `/proc` и `/sys`.


```
[Unit]
Description=AI PC Guardian Agent
After=network-online.target

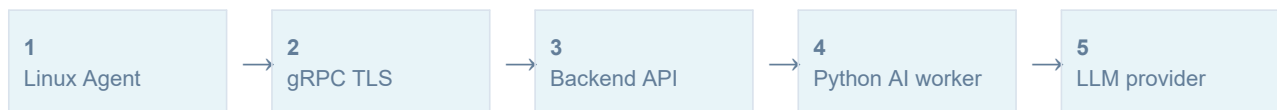
[Service]
Type=simple
ExecStart=/usr/bin/ai-pc-guardian-agent
Restart=on-failure
RestartSec=5s
User=guardian
Group=guardian
NoNewPrivileges=true
ProtectSystem=strict
PrivateTmp=true
StateDirectory=ai-pc-guardian

[Install]
WantedBy=multi-user.target
```

Не запускай от root по умолчанию

Сначала собери минимальный набор непривилегированных метрик. Если конкретная функция требует capability, выдели ее и документируй риск. Полный root расширяет последствия любой уязвимости IPC или обновления.

15. Где находится Python и LLM



Да, сервер может обращаться к LLM через Python: там богаче SDK и быстрее меняется AI-слой. Но это не означает, что весь backend обязан быть Python. gRPC service может принимать C++ или Python сервис, класть нормализованный snapshot в очередь, а отдельный Python worker строит минимальный prompt и вызывает OpenRouter/провайдера.

На агенте	На сервере	Перед LLM
Сырые counters -> общая модель	Auth, schema validation, rate limit	Агрегация и редактирование PII
Локальные правила безопасности	Хранение snapshots и incidents	Allowlist полей
Никаких provider API keys	Billing и audit	Prompt-injection filters + output schema
Retry/queue ограничены	Выбор модели/режима	Timeout, budget, fallback

LLM не является мониторингом

Сбор, thresholds и аварийные правила должны работать без модели. LLM объясняет закономерности и предлагает действия, но сервер валидирует ответ и не исполняет команды автоматически в версии 0.x.

16. Рекомендуемая структура Linux-модуля

```
class LinuxMetricCollector final : public IMetricCollector {
public:
    Result<CpuMetric> collect_cpu(
        std::chrono::milliseconds sample_interval) override;
    Result<MemoryMetric> collect_memory() override;
    Result<DiskMetric> collect_disk(std::string_view mountpoint);
    Result<NetworkSnapshot> collect_network();
};

namespace linux_detail {
Result<CpuTimes> parse_proc_stat(std::istream&);
Result<MemorySample> parse_proc_meminfo(std::istream&);
Result<ProcessSample> parse_proc_pid_stat(std::string_view);
}
```

Чистые parsers вынеси отдельно от доступа к `/proc`: тогда тесты кормят им fixture-строки без Linux host. Platform library реализует общий `IMetricCollector`, а `TelemetryRunner`, common model, protobuf conversion и gRPC остаются общими для Windows/Linux.

17. Тестирование Linux collector

Уровень	Проверки
Unit parser	Пропущенные/дополнительные поля, whitespace, overflow, malformed number
Unit formulas	Counter reset/wrap, zero elapsed, CPU hotplug, network rate
Fixture	Сохранившиеся <code>/proc/stat</code> , <code>meminfo</code> , <code>PID/stat</code> из разных kernel versions
Integration	Чтение реального <code>/proc</code> , <code>statvfs</code> , Unix socket permissions
Service	SIGTERM, restart, readonly filesystem, log/state directory
Cross-platform	Одинаковый protobuf round trip и validation common model

- CI: Ubuntu LTS + sanitizer build; позже добавить Fedora/Arch smoke test.
- Не мокай все подряд: parser unit tests + небольшой реальный integration test дают лучший баланс.
- Ошибки доступа к одному PID и исчезновение PID не должны проваливать весь snapshot.
- Проверь отсутствие секретов в logs и outbound protobuf.

18. Порядок реализации Linux-версии

- Стабилизировать common model и wire protocol на Windows, чтобы Linux не копировал движущуюся цель.
- Создать `platform/linux` library и fixture-based parsers `/proc/stat` + `/proc/meminfo`.
- Реализовать `LinuxMetricCollector` для CPU/RAM/uptime/disk capacity.
- Добавить network counters, process snapshot и optional hwmon.
- Сделать Unix domain socket transport и Qt GUI reuse.
- Добавить packaging (`.deb` сначала), systemd unit и автообновление позже.
- Проверить backend compatibility: один proto contract для обеих ОС.

Реалистичный scope

Linux-клиент не нужен для первого Windows MVP 0.1. Его разумно начать после стабилизации Windows pipeline и backend contract. Большая часть common/proto/grpc/GUI логики переиспользуется, платформенными остаются collector, IPC details, service/package integration.

19. Чек-лист code review

- Collector не запускает shell и не парсит человекочитаемый CLI output.
- `/proc` parser не зависит от порядка необязательных полей и корректно обрабатывает исчезновение PID.
- `fd`/`FILE`/`DIR`/`socket` закрываются `RAII`-владельцем; `errno` сохраняется сразу.
- Единицы нормализованы в `bytes/milliseconds/percent`; `KiB` и `ticks` не теряются.
- Rates считаются по `monotonic elapsed time` и двум `snapshots`.
- Агент работает без `root`; дополнительные capabilities обоснованы отдельно.
- Unix socket расположен в защищенном `runtime dir`, имеет права и `peer check`.
- Один и тот же `TelemetryEvent` проходит round trip на Windows и Linux.

20. Официальные источники

Основные источники - документация ядра Linux, man-pages project, systemd и XDG specifications.

- [The /proc Filesystem](#) - официальное описание `/proc` и process/kernel data
- [proc_stat\(5\)](#) - CPU и другие kernel counters
- [proc_meminfo\(5\)](#) - поля памяти
- [proc_pid_stat\(5\)](#) - process counters и starttime
- [proc_pid_status\(5\)](#) - именованные поля процесса
- [sysinfo\(2\)](#) - uptime, load и memory summary
- [statvfs\(3\)](#) - статистика файловой системы
- [getifaddrs\(3\)](#) - интерфейсы и адреса
- [rtnetlink\(7\)](#) - routing/net interface messages
- [hwmon sysfs interface](#) - датчики в `/sys/class/hwmon`
- [uname\(2\)](#) - kernel/system identity
- [os-release](#) - идентификация дистрибутива
- [machine-id](#) - семантика и privacy machine ID
- [systemd.service](#) - service unit semantics
- [systemd.exec](#) - sandbox и execution environment
- [XDG Base Directory](#) - config/data/state/cache/runtime directories
- [unix\(7\)](#) - Unix domain sockets и credentials
- [signal\(7\)](#) - модель сигналов